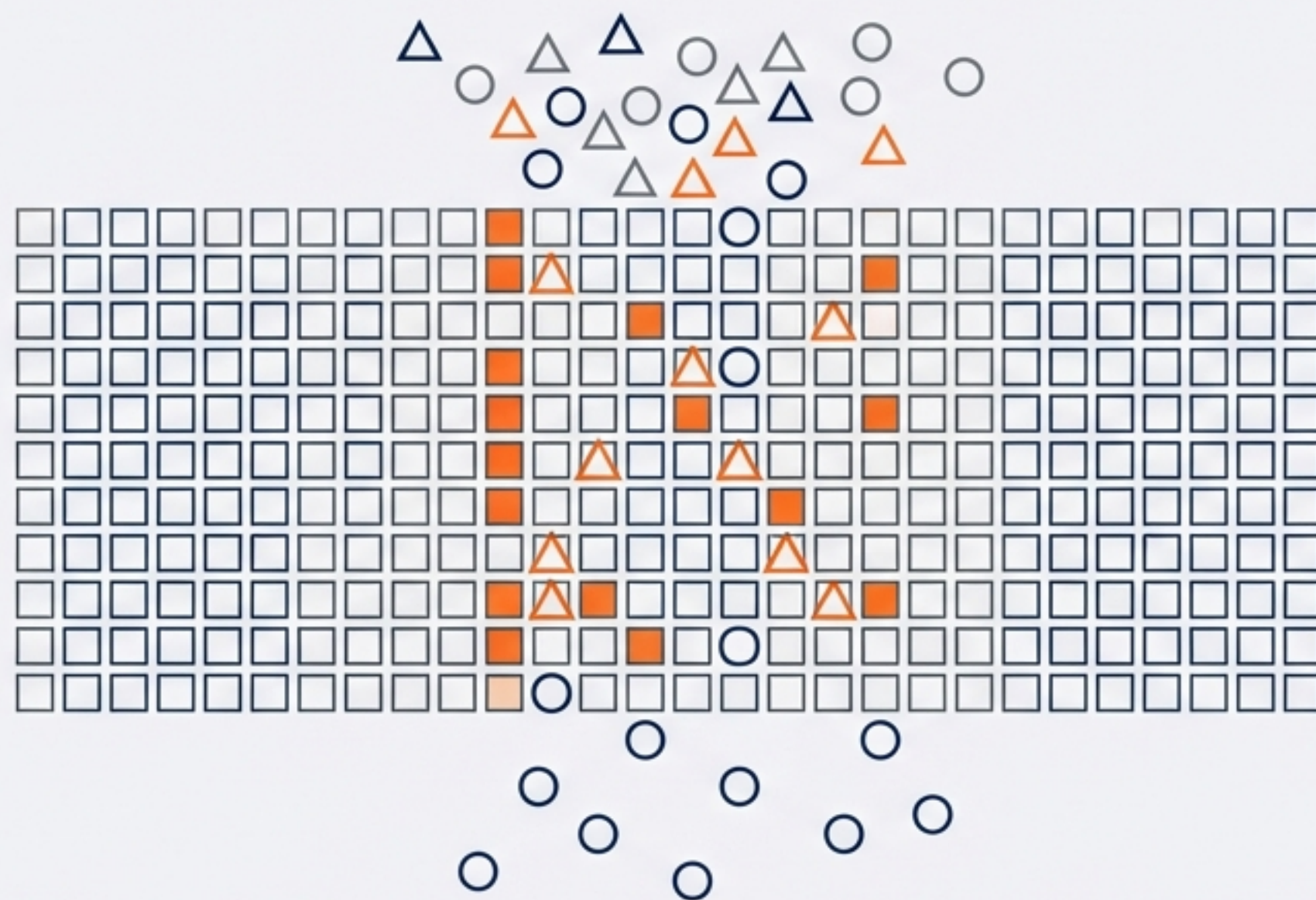


# The Probabilistic Gatekeeper

## Mastering Bloom Filters: From Bitcoin Protocols to Big Data Efficiency

*Mechanisms, Critical Flaws, and Future Architectures.*



1970: Bloom Filter Theory

2012: Bitcoin BIP 37

2018: Neutrino (GCS)

2021: Ribbon Filters



# The Cost of Certainty in a 1.1 Billion GB/Day World

## The Bottleneck

Data generation is accelerating. India alone is projected to hit 1.1 billion gigabytes of data per day by 2025.

Checking set membership—verifying if a specific item exists in these massive datasets—is computationally expensive.

- We cannot scan every SSTable on a disk.
- We cannot download a 200GB+ blockchain for every transaction.
- Traditional lookups require expensive I/O or massive RAM.

## The Probabilistic Shortcut

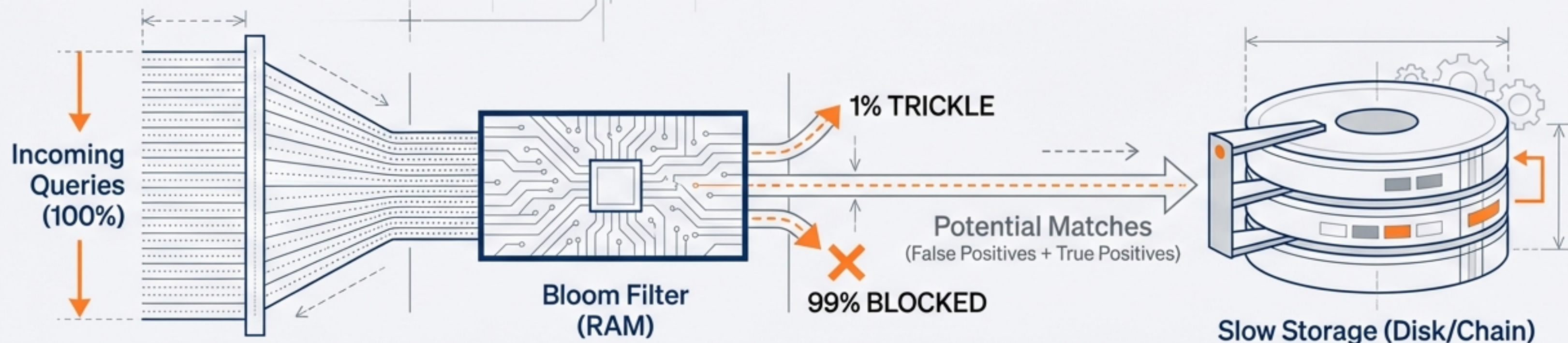
The Bloom Filter offers a space-efficient compromise. It creates a memory-light structure that sits in front of the slow storage.

### Golden Rule:

#### NO FALSE NEGATIVES:

If the filter says **NO**, the data is **definitely not there**. (Skip the disk read).

If the filter says **YES**, the data **MIGHT be there**. (Check the disk to verify).





# Under the Hood: The Hash and the Array

```
class BloomFilter:
```

```
    def add(self, item):
```

```
        # Hash item k times
```

```
        indices = get_hashes(item, k)
```

```
        for index in indices:
```

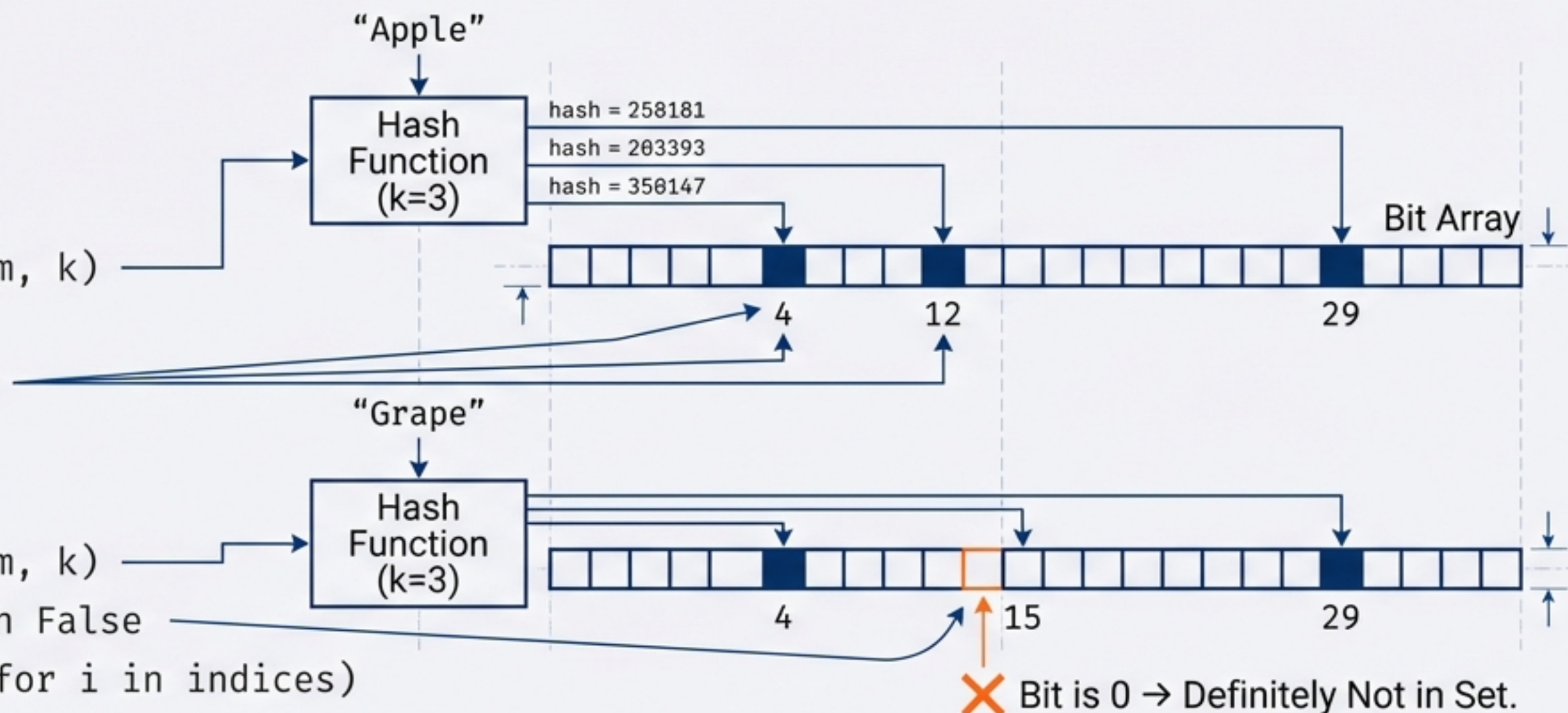
```
            bit_array[index] = 1
```

```
    def check(self, item):
```

```
        indices = get_hashes(item, k)
```

```
        # If any bit is 0, return False
```

```
        return all(bit_array[i] for i in indices)
```



## Optimization Physics

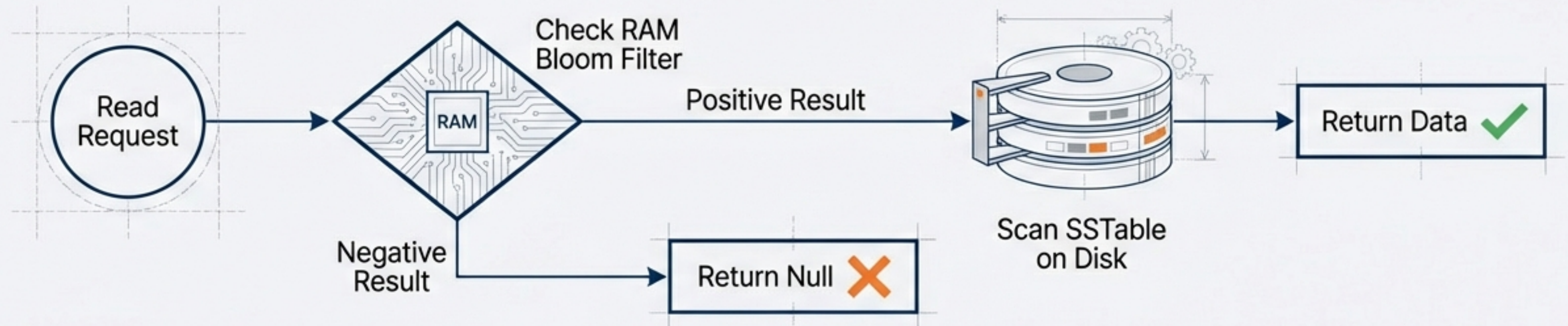
To carry the maximum amount of information per bit (entropy), the bit array should be 50% full.

$$k = (m/n) \ln 2$$



# Path A: The Database Triumph (Apache Cassandra)

## Trading RAM for I/O Operations



## Tuning Dashboard

### Config 1: High RAM / Slow Disk

bloom\_filter\_fp\_chance = 0.01



Virtual elimination of wasted disk I/O. Costs 3x RAM.

### Config 2: Low RAM / Fast NVMe

bloom\_filter\_fp\_chance = 0.1



Saves memory. Accepts occasional wasted seek.

**Key insight:** Cassandra filters are stored offheap, avoiding Java Heap limits.

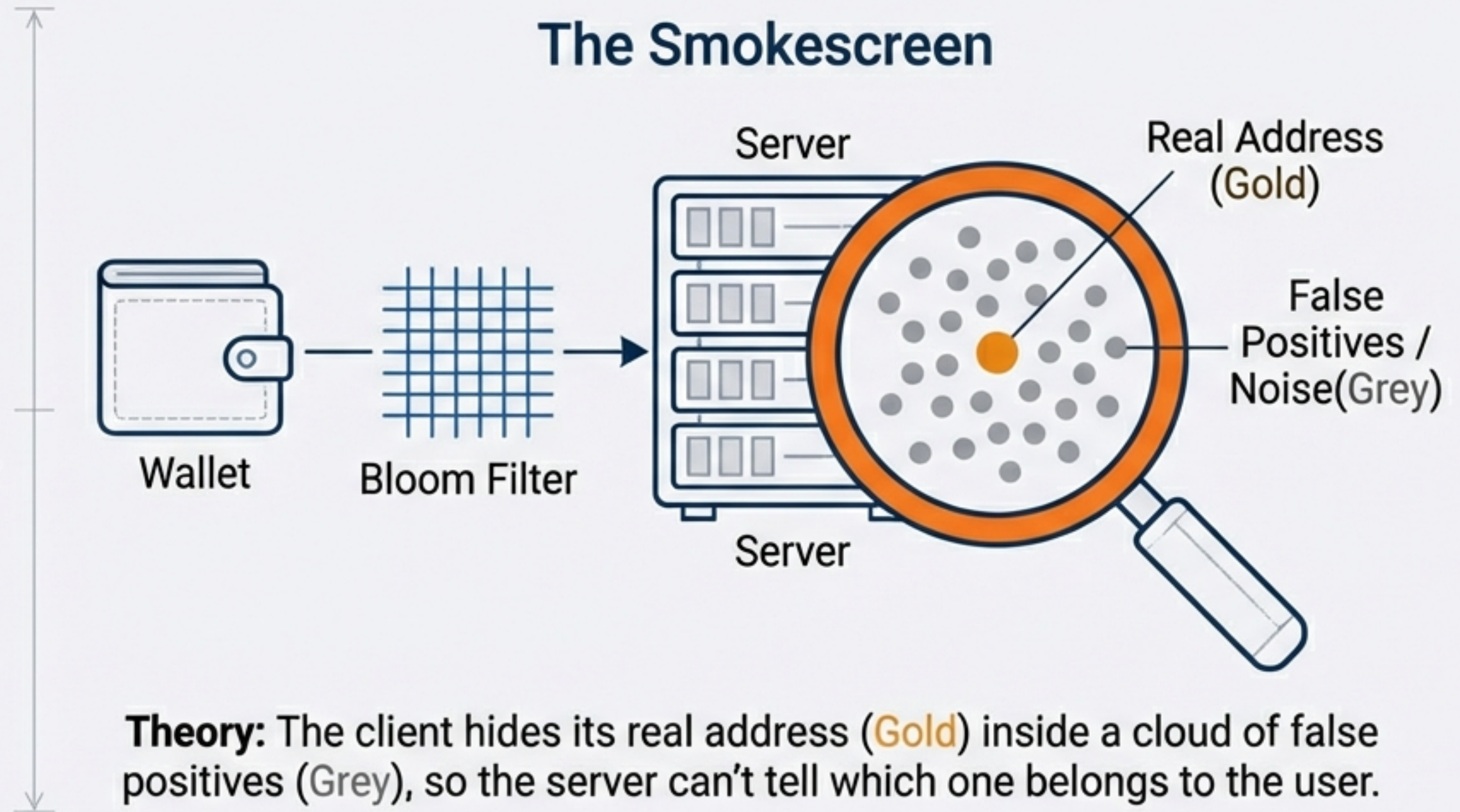


# Path B: The Privacy Trap (Bitcoin BIP 37)

## The Smokescreen Metaphor

In 2012, smartphones needed to track Bitcoin payments without downloading the full blockchain (SPV).

**Solution:** The Light Client (Phone) sends a Bloom Filter of its addresses to the Full Node (Server). The Node only sends back transactions that match the filter.

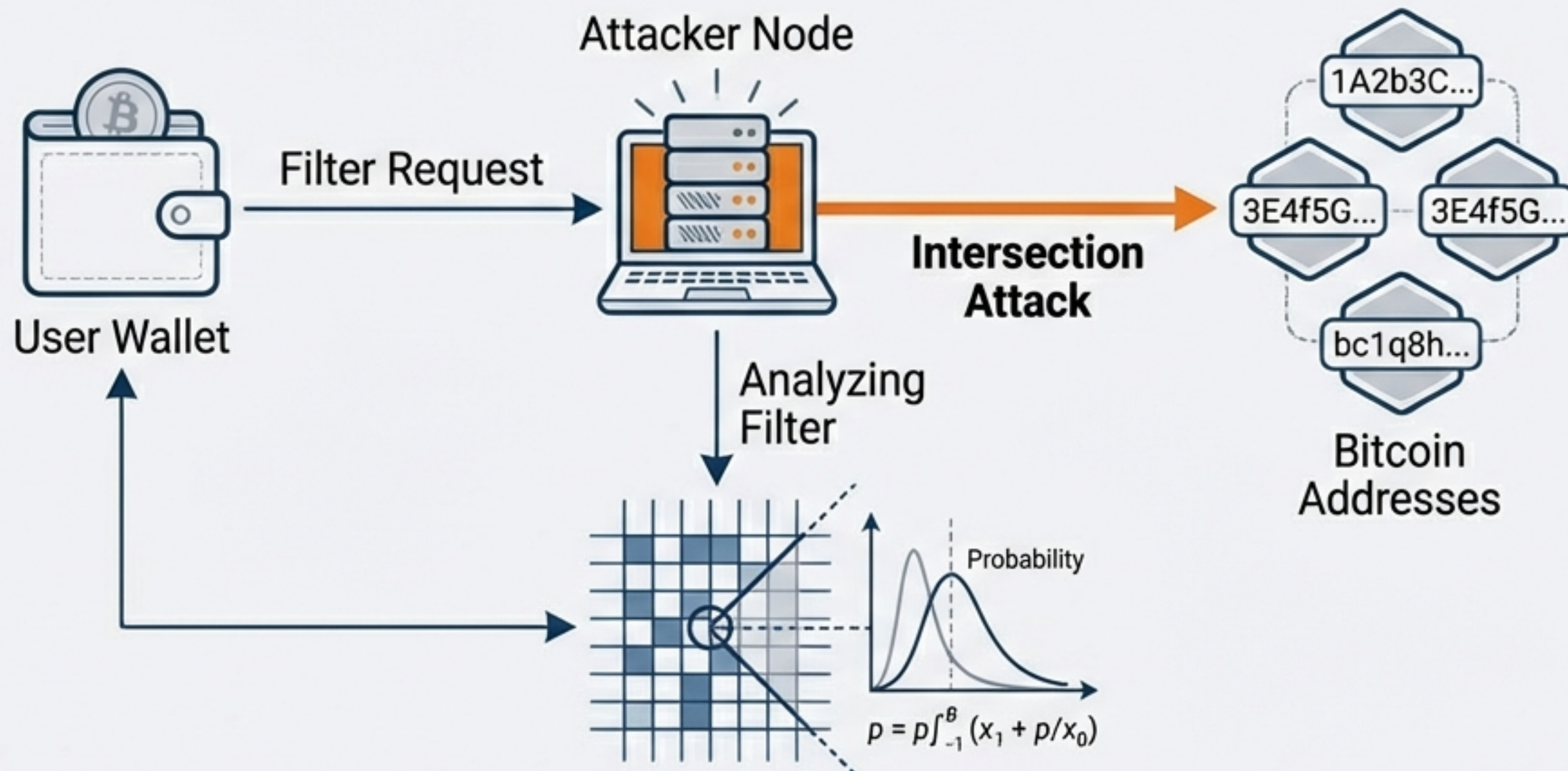


**The Goal:** Bandwidth Efficiency + Plausible Deniability.



# Why BIP 37 Failed: The Deanonymization Attack

## The Attack Vector



## The Privacy Leak

Malicious nodes analyze the probability of the false positives. By checking which transactions a wallet requests over time, they can statistically link all addresses to a single identity.

## The DoS Risk

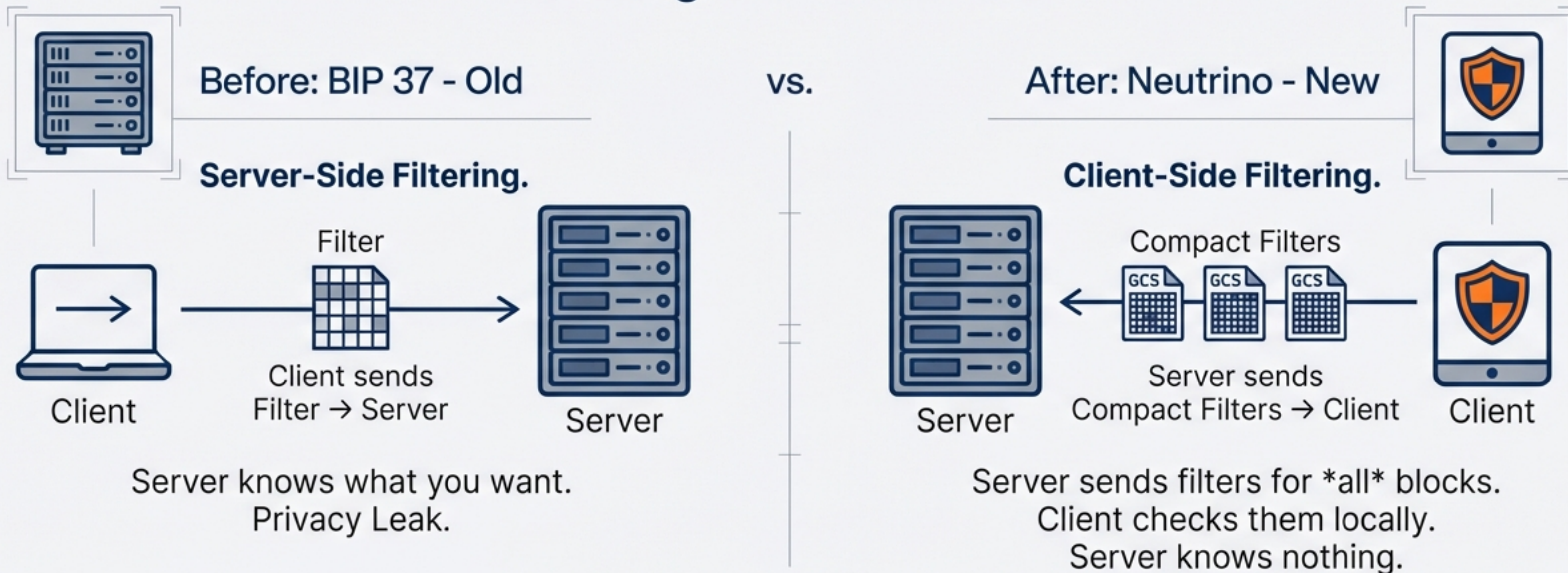
BIP 37 forces the Server to compute the filter for every block. Attackers can craft malicious filters that require massive CPU cycles, crashing the Full Nodes.

**DEPRECATED / OBSOLETE**



# Privacy Solved: The Neutrino Protocol (BIP 157/158)

## Inverting the Architecture



### Technology: **GCS Filters (Golomb-Coded Sets)**

- Compressed filters (~15KB per block).
- The Trade-off: Bandwidth increases (~70MB/month) in exchange for **absolute privacy**.



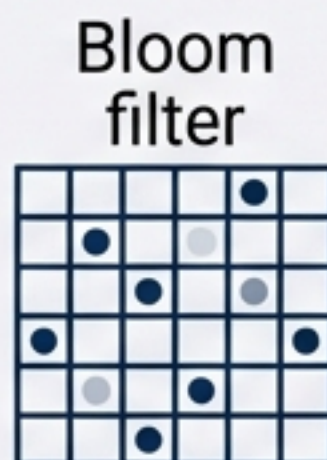
# Beyond Bloom: The Ribbon Filter

Optimizing for Static, Immutable Data

## The Problem

Standard Bloom filters are fast but wasteful. They have >44% space overhead above the theoretical entropy limit.

For massive archival storage (like RocksDB SSTables), this overhead is costly.



## Performance Callout

Space Overhead:  
**Bloom Filter:**  
**Ribbon Filter:**

**~44%**  
**< 1%**

## The Solution

### Gaussian Elimination

|   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 |

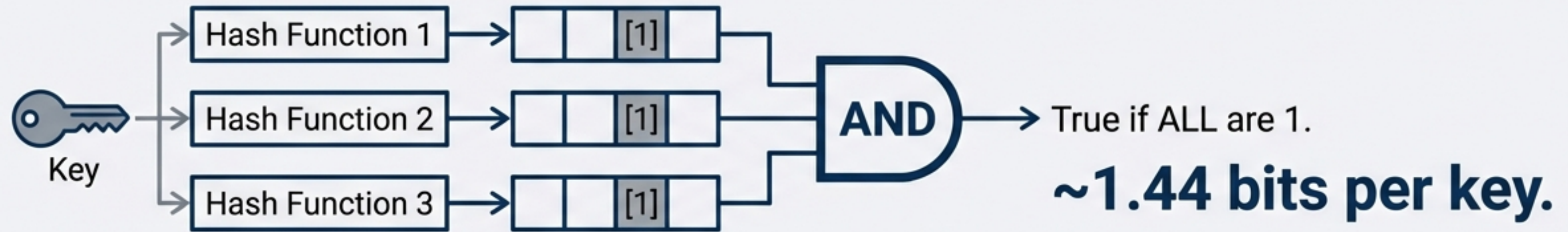
The Ribbon

Instead of setting bits, we solve a linear system of equations (XOR). Constraining the matrix to a narrow ribbon makes the math computationally feasible.

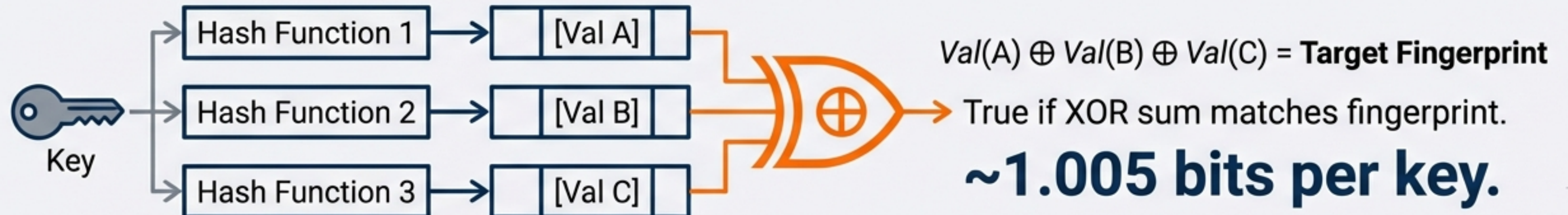


# Mechanism: XOR Probing vs. AND Probing

## AND Probing (Bloom)



## XOR Probing (Ribbon)

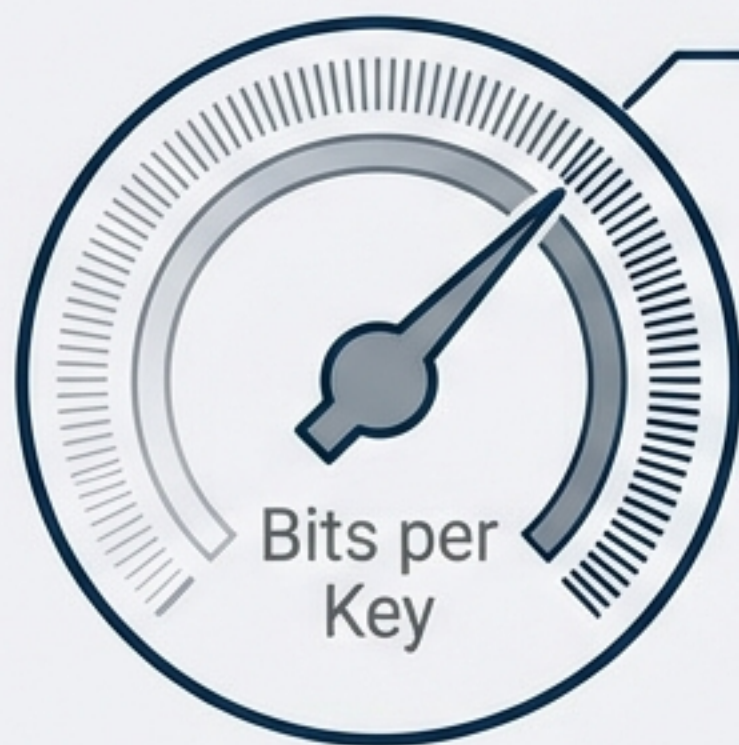


Ribbon/XOR filters are ideal for write-once data. They generally do not support dynamic updates.



# Breaking Constraints: Rational and Block Filters

## Rational Filters (The Math)



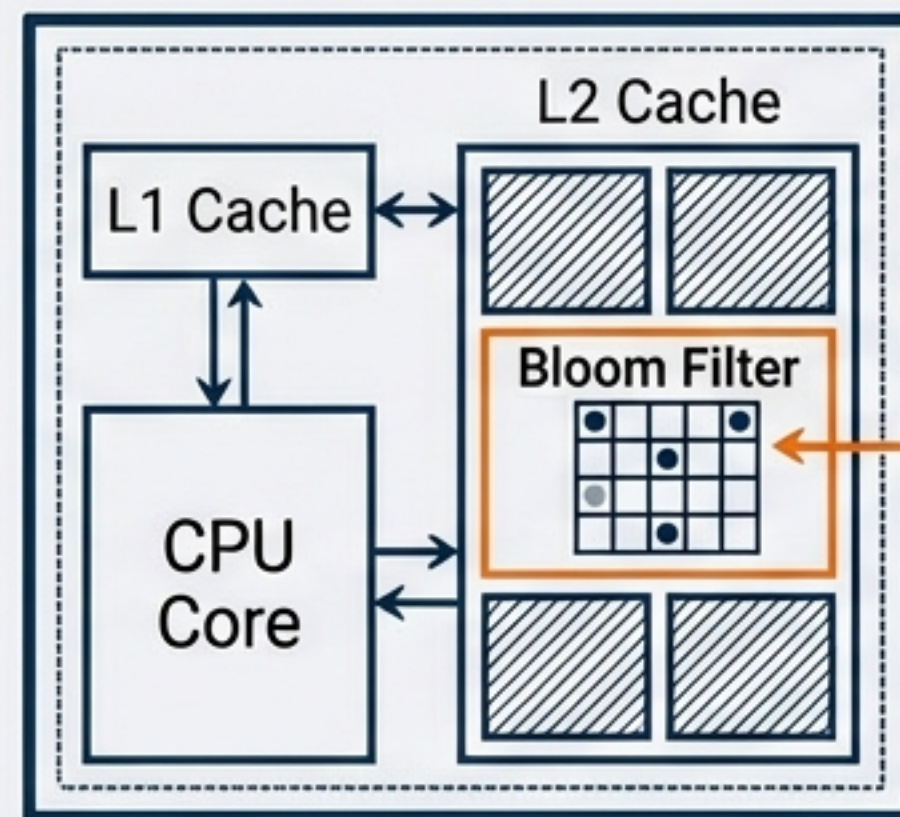
Standard Bloom filters require integer hash counts ( $k$ ).

Rational filters allow non-integer values, enabling granular sizing to fit exact memory pages.

### Performance Callout

**Result: Optimized "fraction-of-zero" values.**

## Block Bloom Filters (The Hardware)



Splitting the filter into small blocks that fit entirely in CPU cache lines to minimize expensive RAM access.

### Performance Callout

**Result: Extreme lookup speed via Cache Locality.**

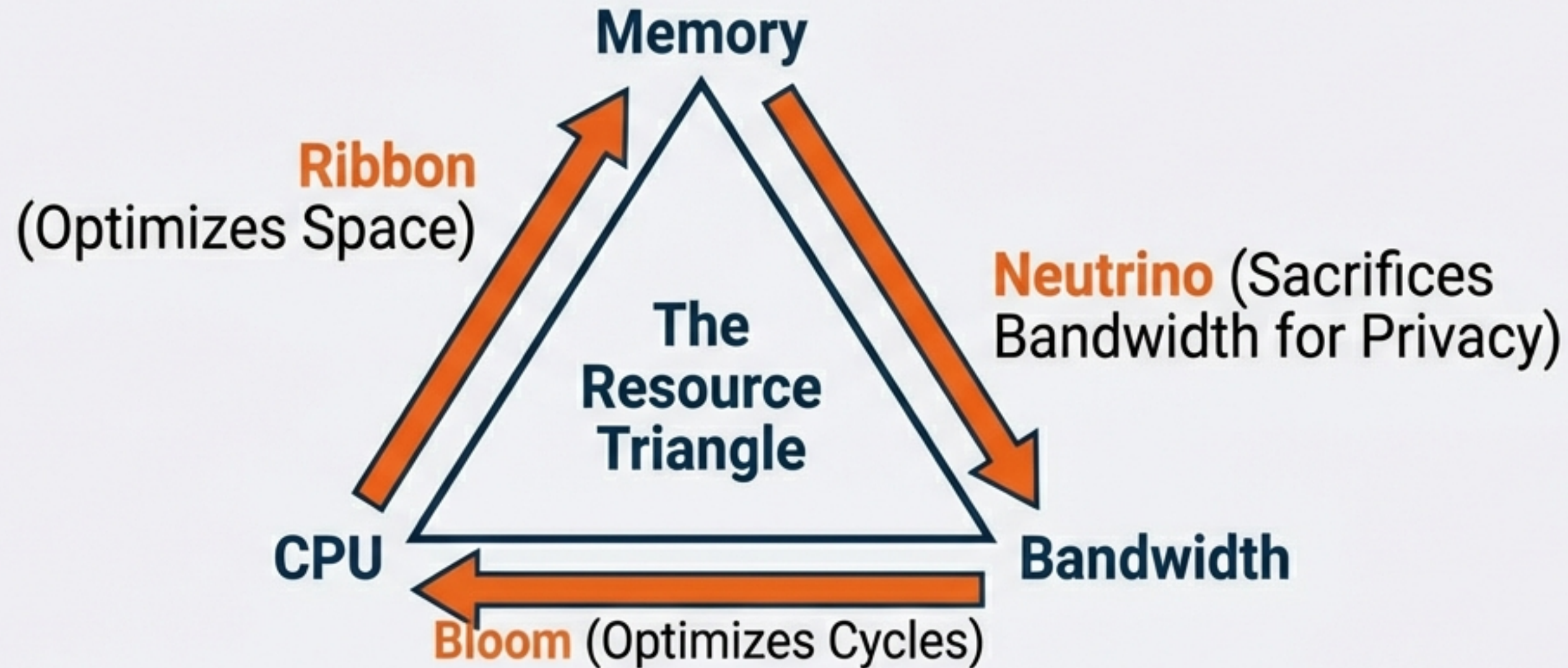


# The Architect's Decision Matrix

| Scenario                       | Recommended Filter           | Why?                                                  | Trade-off                            |
|--------------------------------|------------------------------|-------------------------------------------------------|--------------------------------------|
| General Purpose / Dynamic Data | <b>Standard Bloom Filter</b> | Fast construction, simple, allows deletes (counting). | High space overhead.                 |
| Static / Archival (RocksDB)    | <b>Ribbon / XOR Filter</b>   | Lowest memory footprint (<1% overhead).               | Slower construction, immutable.      |
| Client-Side Privacy (Crypto)   | <b>Neutrino (GCS)</b>        | Preserves user query privacy.                         | Higher bandwidth usage.              |
| Extreme Read Speed             | <b>Block Bloom Filter</b>    | CPU Cache optimization.                               | Slightly higher false positive rate. |



# The Future of Probabilistic Filtering



Filtering is no longer a one-size-fits-all problem. The future is moving toward **Client-Heavy Privacy** (Neutrino) and **Mathematically Optimized Storage** (Ribbon).

**SPV**: Simplified Payment Verification. **GCS**: Golomb-Coded Sets. **Merkleblock**: Filtered block containing only matching transactions.  
Sources: Lightspark, Apache Cassandra, Facebook Engineering (Ribbon), arXiv.